

EMCALI Cyber 2.0

Arquitectura por capas, diagrama técnico y tabla comparativa

Proyecto: Agente Consultor en Ciberseguridad con LLMs Locales

Enfoque: Qualys/Tenable + MCP/FastMCP + LangGraph/LangChain + Ollama/vLLM + RAG

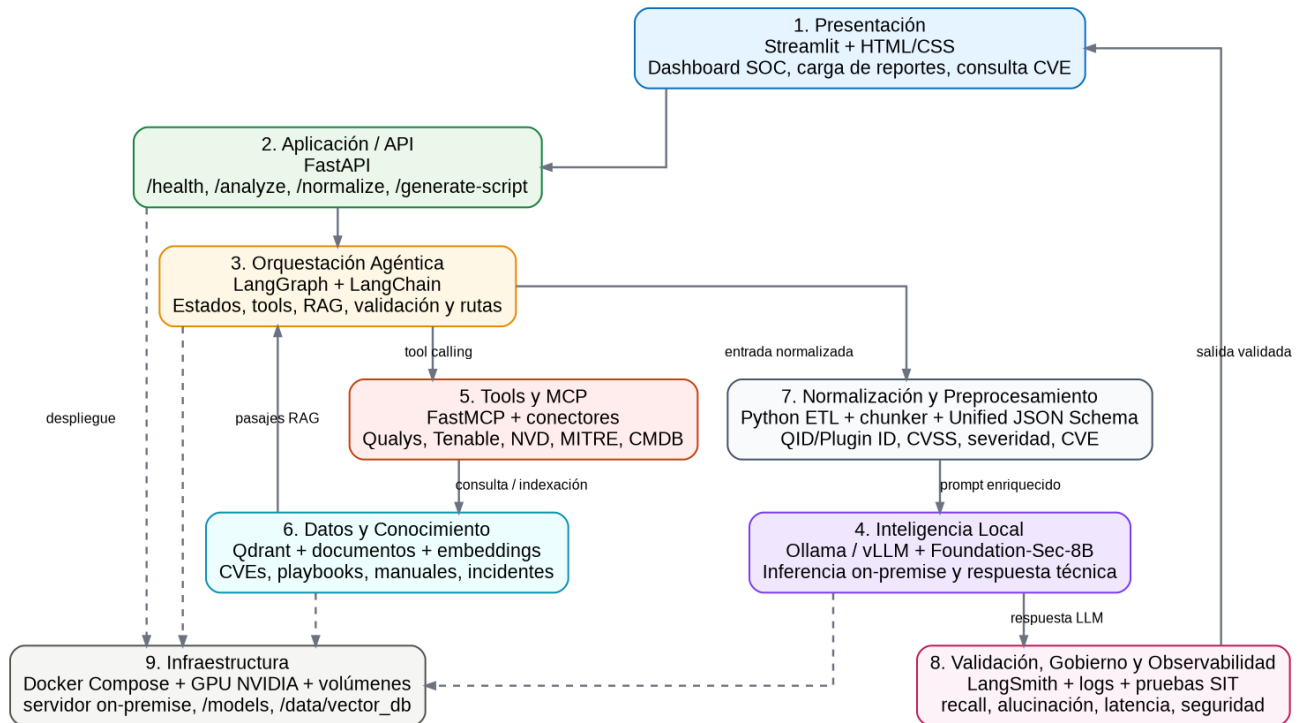


Figura 1. Arquitectura general por capas propuesta para EMCALI Cyber 2.0.

1. Resumen ejecutivo

EMCALI Cyber 2.0 se plantea como una solución local-first para análisis de vulnerabilidades asistido por inteligencia artificial. Su núcleo operativo combina modelos locales ejecutados con Ollama o vLLM, una capa de orquestación agéntica basada en LangGraph y LangChain, conectores de herramientas mediante MCP/FastMCP, una base vectorial para RAG y una capa de validación orientada a seguridad, métricas y trazabilidad.

La primera fase debe concentrarse en el análisis de vulnerabilidades provenientes de Qualys y Tenable, con procesamiento de logs como segunda iteración. Esta decisión está alineada con el acta del proyecto, donde se priorizó el análisis de vulnerabilidades, la comparación de modelos, el uso de Ollama, y la evaluación de MCP servers como estrategia más escalable que la carga manual de CSV.

La arquitectura propuesta evita depender únicamente del prompt. En su lugar, organiza el sistema como una cadena de capas: fuentes de datos, normalización, esquema unificado, orquestación, RAG, inferencia local, validación y presentación. Esta separación reduce alucinaciones, mejora auditabilidad y facilita el crecimiento hacia nuevos casos de uso del SOC.

2. Principios de diseño

Principio	Implicación técnica
Soberanía de datos	Los hallazgos de vulnerabilidades, activos, IP internas, playbooks y evidencias deben procesarse localmente salvo autorización expresa para escenarios híbridos.
Arquitectura tool-driven	El agente debe consultar herramientas y fuentes verificables en lugar de depender de datos pegados manualmente en el prompt.
Agentes especializados	Se recomienda separar funciones: priorización, explicación técnica, generación de scripts, enriquecimiento CVE y resumen ejecutivo.
Normalización previa al LLM	Qualys y Tenable usan esquemas distintos; el LLM debe recibir un Unified JSON Schema estable.
Evaluación medible	La aceptación debe basarse en recall, tasa de alucinación, latencia, robustez, exactitud de IOC's y validación de scripts.
Despliegue reproducible	Docker Compose, volúmenes persistentes y GPU dedicada permiten trazabilidad y recuperación operativa.

3. Arquitectura por capas

La arquitectura por capas permite separar responsabilidades, reducir acoplamiento y facilitar la evolución del sistema. Cada capa cumple una función clara dentro del ciclo de análisis de vulnerabilidades.

Capa	Componentes	Responsabilidad	Resultado
1. Presentación / Interfaz	Streamlit, HTML/CSS, dashboards SOC	Carga de reportes, consulta CVE, visualización de recomendaciones, descarga de informes y scripts.	Interfaz unificada para analistas y equipo TI.

2. Aplicación / API	FastAPI	Exponer endpoints /health, /analyze, /normalize, /generate-script, /ingest-docs.	Servicio desacoplado y reutilizable.
3. Orquestación agéntica	LangGraph, LangChain, LangSmith	Gestionar estado, seleccionar tools, consultar RAG, componer prompt final, validar salida.	Agentes especializados y trazables.
4. Inteligencia artificial local	Ollama o vLLM + Foundation-Sec-8B	Ejecutar inferencia on-premise para análisis, priorización y generación de texto/código.	Respuesta técnica sin fuga de datos sensibles.
5. Tools / MCP	FastMCP, servidores MCP, conectores API	Exponer funciones como get_vulnerabilities, lookup_cve, search_playbook, get_asset_details.	Integración estandarizada con fuentes y sistemas.
6. Datos y conocimiento	Qdrant, embeddings, repositorio documental	Almacenar CVEs, manuales, playbooks, bitácoras, incidentes y pasajes de conocimiento.	Memoria de largo plazo para RAG.
7. Normalización y preprocesamiento	Python ETL, chunker, Unified JSON Schema	Mapear QID/Plugin ID, severidad, CVSS, CVE, evidencia, activo y contexto.	Entrada limpia y consistente para el modelo.
8. Validación y gobierno	LangSmith, logs, validators, pruebas SIT	Medir alucinación, recall, latencia, sintaxis de scripts, seguridad de tool calls.	Salida confiable, auditable y apta para revisión.
9. Infraestructura	Docker, GPU NVIDIA, volúmenes persistentes, servidor on-premise	Ejecutar contenedores, modelos, bases vectoriales y servicios internos.	Plataforma local estable y reproducible.

4. Diagrama lógico y flujo operativo

El flujo operativo recomendado evita enviar reportes completos al modelo. Las fuentes se exponen mediante tools/MCP, se normalizan a un esquema unificado y luego el orquestador decide si debe consultar RAG, invocar el modelo o devolver la respuesta a validación.

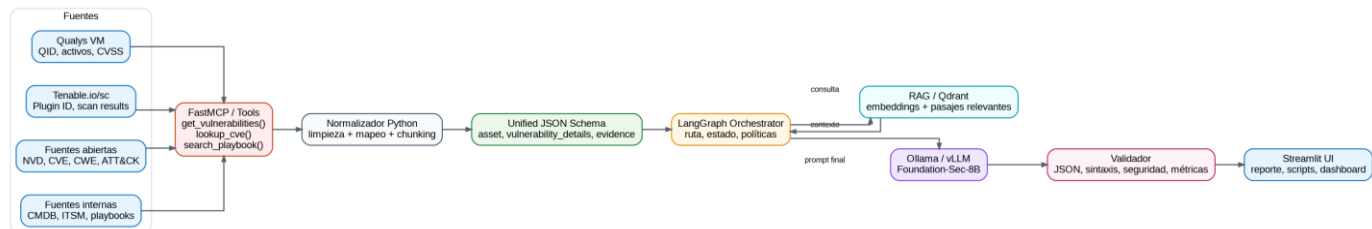


Figura 2. Flujo operativo: fuentes, MCP, normalización, RAG, LLM y salida validada.

5. Arquitectura de despliegue on-premise

Para la versión local, se recomienda separar contenedores por responsabilidad: interfaz, backend, servicio agéntico, gateway MCP, runtime LLM y base vectorial. Los modelos y los índices deben persistirse en volúmenes dedicados.

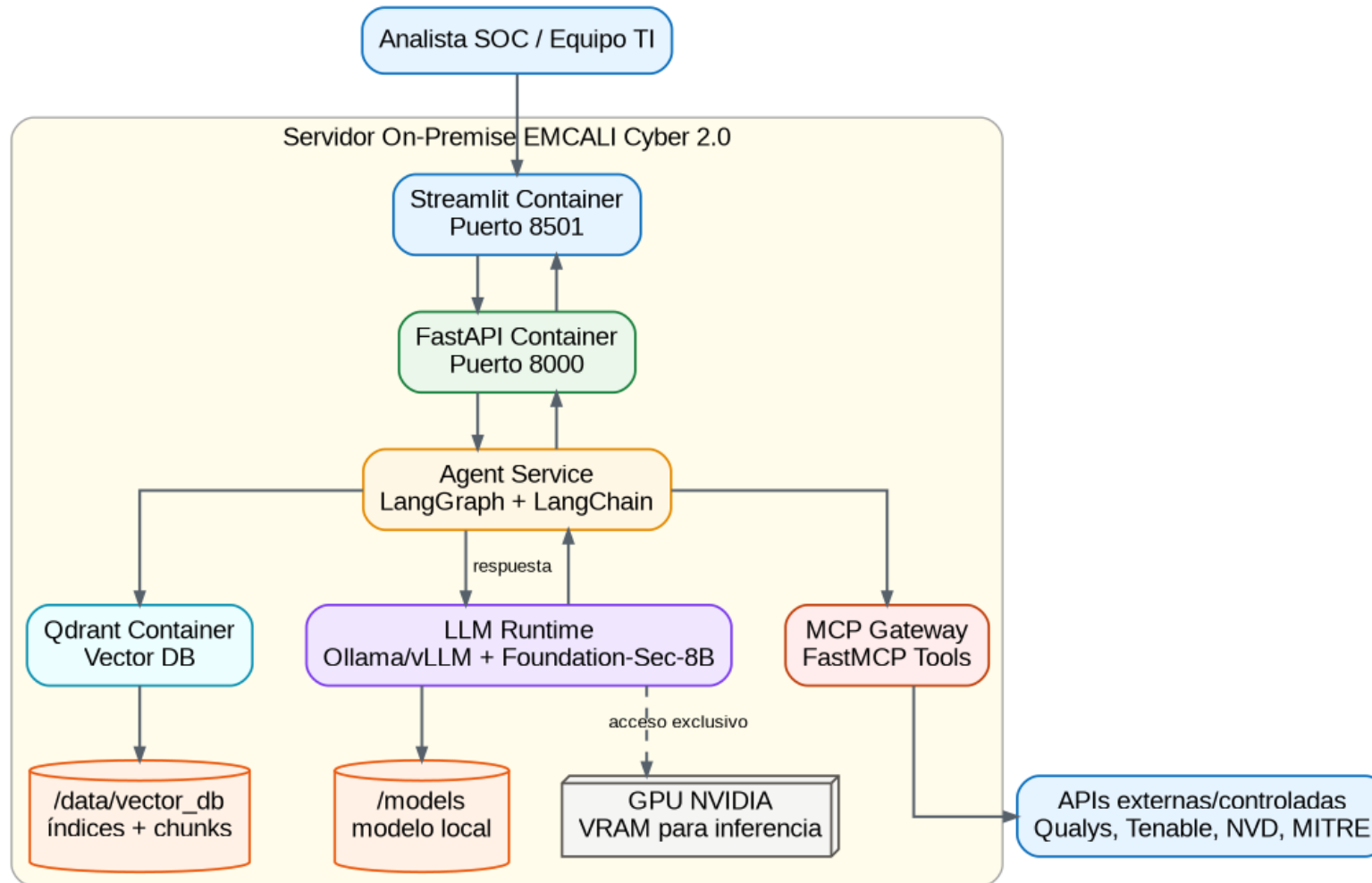


Figura 3. Despliegue on-premise con FastAPI, agente, MCP, LLM runtime, Qdrant, GPU y volúmenes.

6. Tabla comparativa de componentes

Componente	Rol	Ventajas	Limitaciones	Uso recomendado
Ollama	Runtime local de modelos abiertos	Instalación simple, API local, privacidad	Menor optimización que vLLM en alta concurrencia	Laboratorio y primera versión local
vLLM	Motor de inferencia de alto rendimiento	Mejor throughput y uso de GPU	Más complejo de desplegar	Producción con mayor carga
LangChain	Framework de integración de agentes	Conecta modelos, tools, retrievers y parsers	Puede volverse desordenado sin arquitectura	Base de integración
LangGraph	Orquestación con estado	Control de rutas, nodos, human-in-the-loop	Requiere diseño explícito del grafo	Núcleo del agente
LangSmith	Observabilidad y evaluación	Trazas, métricas, debugging, evaluación	Puede requerir servicio externo o self-hosting	Gobierno y calidad
FastMCP	Creación de servidores MCP	Expone funciones Python como tools estandarizadas	Requiere seguridad y contratos de tool claros	Conectores Qualys/Tenable/NVD/CMDB
Qdrant	Base vectorial para RAG	Búsqueda semántica rápida y local	Requiere curaduría documental	Memoria de conocimiento
n8n	Automatización visual periférica	Workflows rápidos, aprobaciones, integraciones	No reemplaza el orquestador agéntico	Complemento operativo
OpenAI Agent Builder	Diseño visual de agentes	Prototipado y exportación de flujos	No es núcleo local-first	Benchmark/laboratorio
Microsoft Foundry	Plataforma empresarial de agentes	Gobierno cloud, conectores, escalabilidad	Dependencia Azure/cloud	Escenario híbrido futuro

7. Hoja de ruta de implementación

Fase	Actividades	Entregable
Fase 0 - Preparación	Instalar Ollama/vLLM, Docker, GPU drivers, repositorio base, variables seguras.	Entorno local listo.
Fase 1 - Ingesta y normalización	Implementar parsers Qualys/Tenable y Unified JSON Schema.	Datos limpios y homogéneos.
Fase 2 - Agente mínimo viable	Construir LangGraph con nodos: clasificar, normalizar, consultar RAG, inferir y validar.	Agente funcional en Streamlit/FastAPI.
Fase 3 - MCP/FastMCP	Exponer tools de consulta a APIs, CVE, playbooks y CMDB.	Arquitectura tool-driven.

Fase 4 - RAG local	Indexar CVEs, manuales, playbooks y documentos internos en Qdrant.	Respuestas enriquecidas y verificables.
Fase 5 - Validación y gobierno	Agregar LangSmith/tracing, métricas, pruebas SIT y validadores de scripts.	Control de calidad y evidencia.
Fase 6 - Automatización periférica	Conectar n8n para reportes programados, aprobaciones y notificaciones.	Operación extendida.

8. Controles mínimos de seguridad y gobierno

- Usar vault o variables protegidas para API keys de Qualys, Tenable, NVD y servicios internos.
- Aplicar listas blancas de tools permitidas y bloquear ejecución de comandos arbitrarios desde el LLM.
- Registrar trazas de cada decisión del agente: entrada, tools llamadas, pasajes RAG, prompt final y salida.
- Agregar revisión humana obligatoria para scripts de remediación, cambios en producción y hallazgos críticos.
- Probar resistencia a prompt injection, fuga de información y tool misuse antes de producción.
- Medir alucinación técnica y falsos negativos con dataset de prueba controlado.

9. Fuentes base y referencias técnicas

- Acta de reunión 30 de marzo de 2026: definición de primera fase, uso de Ollama, análisis Qualys/Tenable, evaluación MCP y agentes especializados.
- Actividad 1 GLM-4.7: métricas de evaluación, normalización Qualys/Tenable, Unified JSON Schema, MCP vs CSV, Foundation-Sec-8B, RAG y pruebas SIT.
- Ollama: ejecución local de modelos abiertos y API local.
- LangChain / LangGraph / LangSmith: agentes, tools, orquestación con estado, trazabilidad y evaluación.
- Model Context Protocol / FastMCP: exposición estandarizada de tools, recursos y datos a aplicaciones LLM.
- NVD, MITRE ATT&CK, CWE: fuentes abiertas para enriquecimiento de vulnerabilidades y contexto adversarial.
- n8n, OpenAI Agent Builder, Microsoft Foundry: plataformas complementarias para automatización, prototipado o gestión empresarial de agentes.